
Group 5

Orphan-care Final Report

Version 1.0

PID:15

10/10/2024

Group Members:

Gamage M.S.210176F
Gunasekara S.L. 210194H
Gunathunga W.S.D. 210196P

Abstract/ Executive summary

This report addresses a significant challenge faced by the Child Protection Authority: the reliance on an outdated manual system for managing orphanages and the sensitive data of children in their care. The goal of this project is to develop an advanced Management Information System (MIS) that emphasizes data privacy and restricts access to authorized personnel. The system ensures secure access to orphanage data through role-based permissions. Additionally, it offers data analytics and reporting capabilities, enabling effective monitoring of orphanage performance. A key feature of the system is its automatic handling and ensuring robust security. This report outlines the goals, methodologies, and potential impacts of this innovative MIS, which aims to transform child protection efforts by improving data management and security.

Table of Contents

1. Introduction
 - 1.1 background of the application domain/ problem
 - 1.2 motivation for the selected system development
 - 1.3 importance and main purpose of the system
 - 1.4 overview/ summary of the system and used approach and outcome
2. Literature Review
3. System Models
 - 3.1 System Requirement
 - 3.2 System Design
 - 3.3 Database Design
4. System Implementation
 - 4.1 Implementation Procedure
 - 4.2 Materials
 - 4.3 Main Interfaces
5. System Testing and Analysis
 - 5.1. Testing approach
 - 5.2. Unit Testing, Results and analysis of testing
 - 5.3. Aspects related to performance, security, failures
 - 5.4. System Usability Report
6. Conclusion and Future Work
- Reference

1. Introduction

1.1 Background of the application domain/ problem

The development of the OrphanCare Data Privacy Management Information System (MIS) represents a critical leap forward for the Child Protection Authority in managing orphanages and facilitating the adoption process. This system replaces outdated manual information-handling methods with an integrated digital web application platform. Key advantages include enhanced data privacy, reduction of redundancy, improved data accuracy, and significantly increased efficiency. With a scalable, online design, the system ensures seamless communication among stakeholders, responding effectively to the pressing needs of child protection authorities and improving oversight across orphanage management and adoption workflows.

1.2 Motivation for the selected system development

The motivation for creating this MIS is driven by the need to streamline the complex processes involved in managing orphanages and safeguarding child welfare. Traditional paper-based methods are increasingly inadequate for addressing growing concerns over data privacy, security, and accessibility. The OrphanCare system was developed to meet these challenges head-on, providing child protection authorities with the tools necessary to efficiently manage sensitive information, improve data security, and ensure compliance with privacy standards. This system equips the Child Protection Authority with advanced capabilities to fulfill its mission of protecting vulnerable children by implementing a secure, scalable, and efficient digital approach.

1.3 Importance and main purpose of the system

The significance of the OrphanCare MIS is profound, as it fundamentally reshapes the management of orphanage and child welfare data. Its main goal is to strengthen the protection and well-being of children by enabling quicker access to essential information, improving data accuracy, and safeguarding the privacy of sensitive child-related records. By transforming decision-making processes for child welfare authorities, the system offers comprehensive insights into key child protection metrics, boosting the overall efficiency of child welfare services through enhanced data security and increased operational transparency.

1.4 Overview/ summary of the system and used approach and outcome

The OrphanCare MIS offers a comprehensive, privacy-focused, and technologically advanced solution for the Child Protection Authority to manage orphanage and child welfare data. By leveraging web and mobile applications, the system provides a secure, user-friendly platform that addresses the complexities of managing sensitive information. The approach taken emphasizes scalability, data privacy, and ease of access, ensuring stakeholders can interact with the system efficiently. The outcome is a transformative digital resource that eliminates redundancies, increases data accuracy, and fortifies data privacy, enabling real-time decision-making. By automating key processes, the system significantly improves child welfare management, fostering better outcomes in child protection efforts.

2. Literature Review

OrphanCare is a web application that serves as a management information system (MIS) for a child protection authority. The system will protect children's data while allowing authorized professionals to access and handle necessary information. This system is easily accessible to all users through the web application. It's designed for various users, including Administrator(Child Protection Authority), orphanage Head, orphanage staff, parents, and social workers. The web application's frontend is created with React.js, while the backend relies on Node.js and Express. The system's database operates on PostgreSQL. For secure document storage, AWS cloud is used.

3. System Models

3.1 System Requirement

Functional Requirements

The OrphanCare Data Privacy Management Information System (MIS) encompasses a set of functional requirements designed to effectively meet the needs of all users involved in orphanage management and adoption processes. The platform features a landing page accessible to all users, directing them to a login interface for credential entry. Upon successful authentication, users are redirected based on their roles:

- Administrators gain access to a comprehensive admin dashboard, equipped with tools for managing orphanages, staff, and adoption applications.
- Orphanage heads, social workers, and staff navigate to their specific orphanage management pages, enabling oversight of child profiles and management of adoption workflows.
- General users enter a user dashboard to track their case progress and view relevant information.

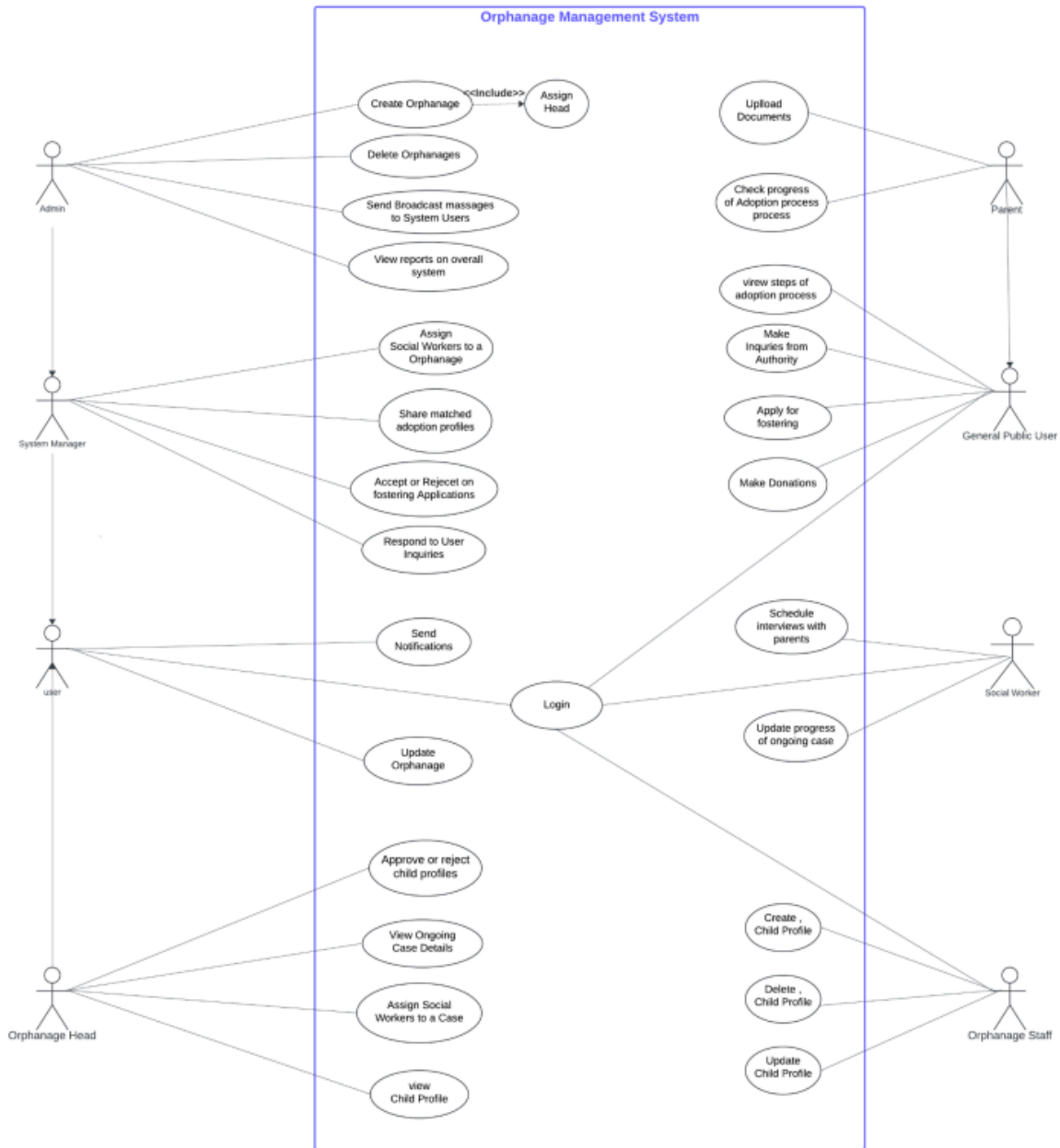
Role-based access control is implemented to ensure that each user can only perform actions pertinent to their designated role, thereby maintaining the integrity and confidentiality of sensitive information. Additionally, the platform supports document uploads, validation processes, and communication channels, including video conferencing, essential for conducting interviews and assessments throughout the adoption process. Overall, these functional requirements create a streamlined, secure, and user-friendly environment that prioritizes child welfare and upholds privacy standards.

Non-Functional Requirements

The OrphanCare Data Privacy Management Information System is designed to meet a broad spectrum of general non-functional requirements essential for its effective operation. A strong emphasis is placed on accessibility and usability, ensuring that the platform remains user-friendly for all stakeholders. Stringent security measures are implemented to safeguard sensitive child protection data, thereby fostering trust and compliance with privacy standards. The system also incorporates robust supportability features to facilitate efficient maintenance and continuous improvement. Additionally, high availability is prioritized to minimize potential downtime, ensuring that users can access the platform whenever needed. This comprehensive approach to non-functional requirements underscores OrphanCare's commitment to delivering an accessible, secure, and reliable environment, with a particular focus on user-friendliness as a primary objective.

User diagram

The use case diagram for the OrphanCare Data Privacy Management Information System outlines various user roles, each assigned specific functionalities within the platform. The main user roles include system administrators, orphanage heads, orphanage staff, prospective adoptive parents, and social workers. The system supports six primary use cases that reflect a hierarchical configuration of user interactions. Orphanage heads and staff share several functionalities related to managing child profiles and overseeing adoption workflows, ensuring collaboration within the orphanage management process. In contrast, prospective adoptive parents and social workers are assigned distinct roles and interactions, focusing on the adoption application process and the assessment of suitability. This structured organization of use cases clearly defines user responsibilities and interactions, facilitating a streamlined approach to orphanage management and the adoption process while prioritizing child welfare.

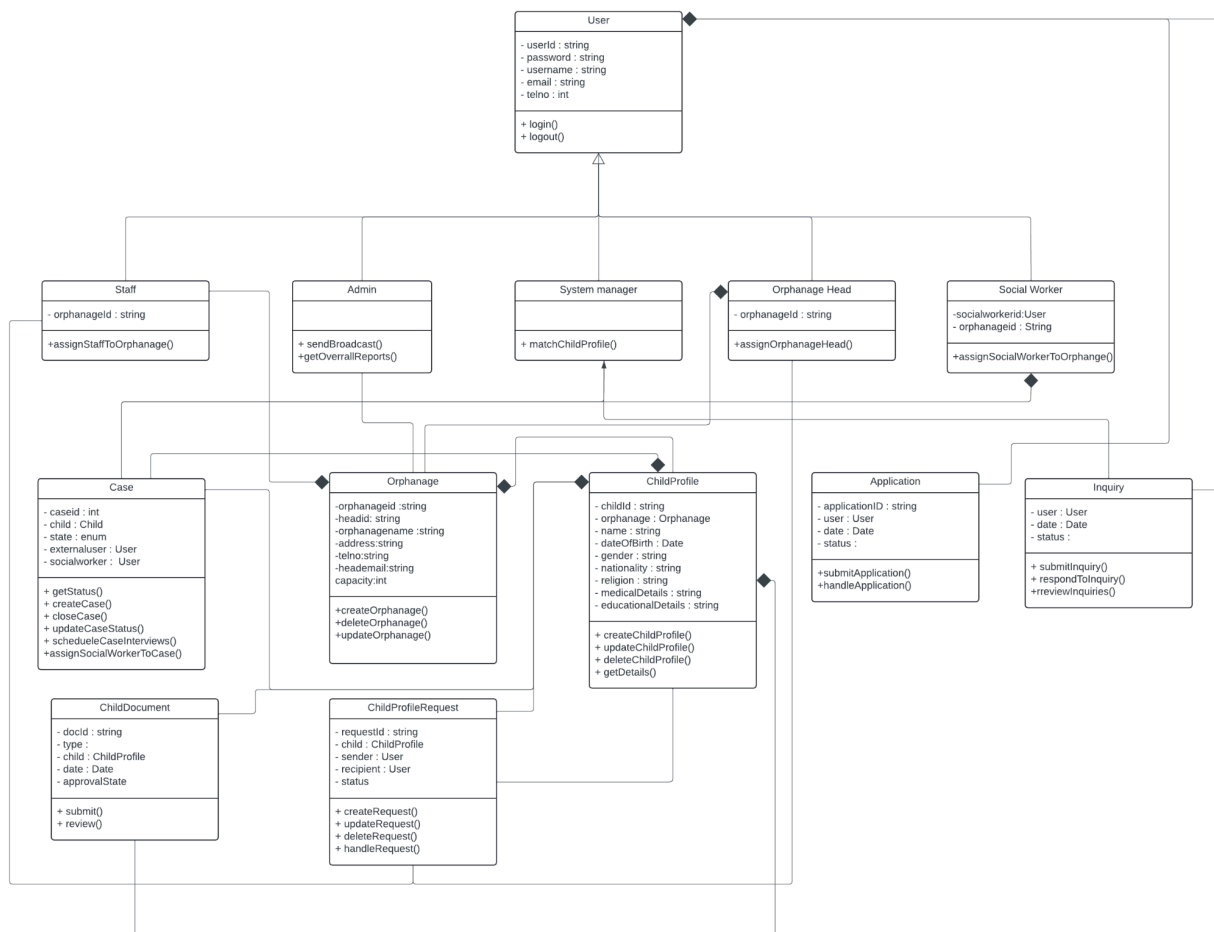


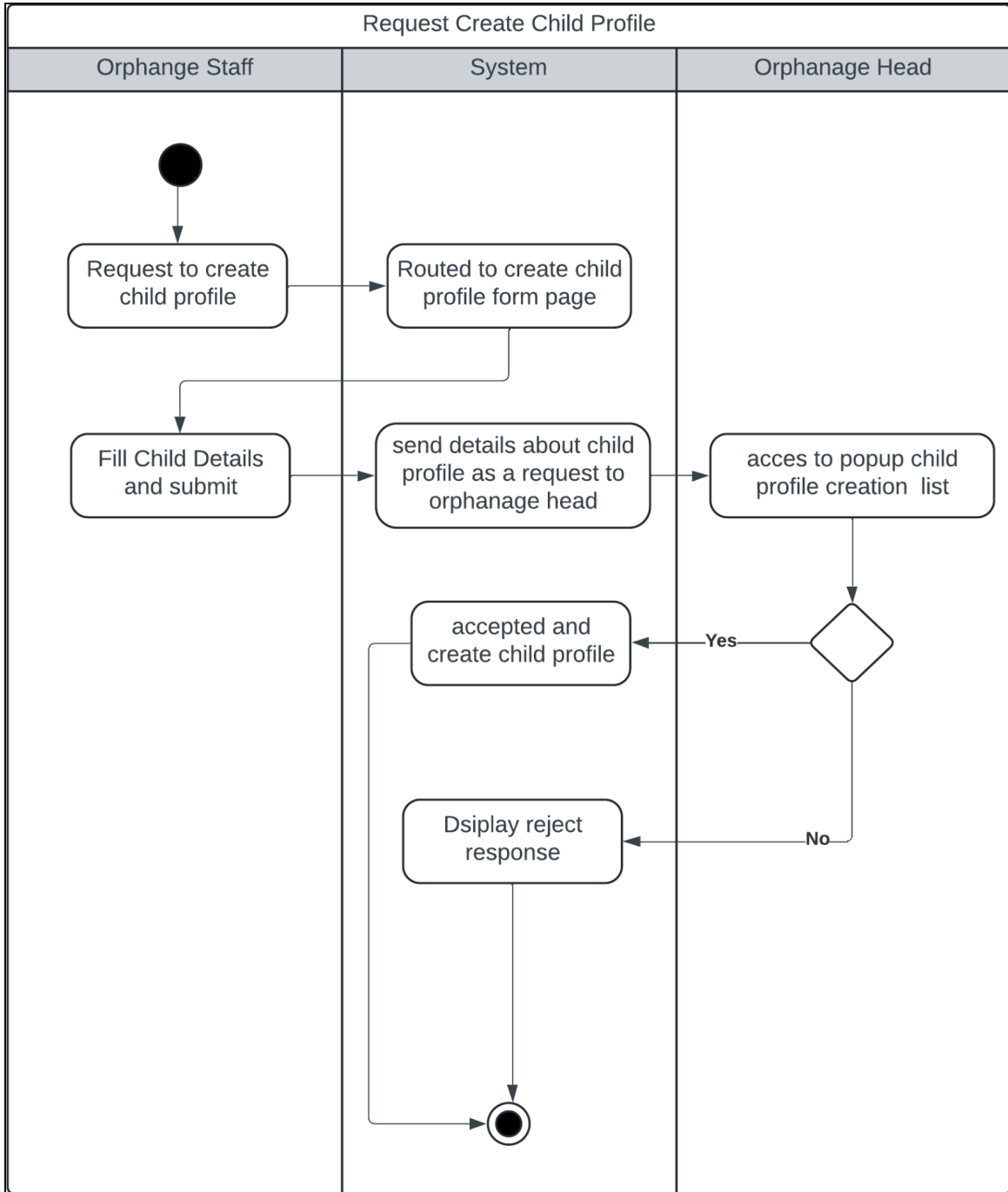
3.2 System Design

The Data Privacy Based MIS for Child Protection Authority follows a 3-tier architecture to ensure scalability, security, and performance. The **Presentation Layer**, built with React JS and Tailwind, provides a responsive and user-friendly interface for users like System Administrators, Orphanage Heads, Social Workers, and the public, facilitating interactions such as child profile management, fostering applications, donations, and inquiries. The **Business Logic Layer**, developed using Node.js and Express.js, handles role-based access control (RBAC), validation, adoption and fostering processes, user communication, and data privacy. The **Data Layer** utilizes PostgreSQL for securely storing orphanage, child, and adoption data, while AWS S3 is used for document storage, including adoption-related files and child profiles. Monitoring tools like AWS CloudWatch to track system health and performance. The proposed class diagram is shown below.

Request to Create Child Profile

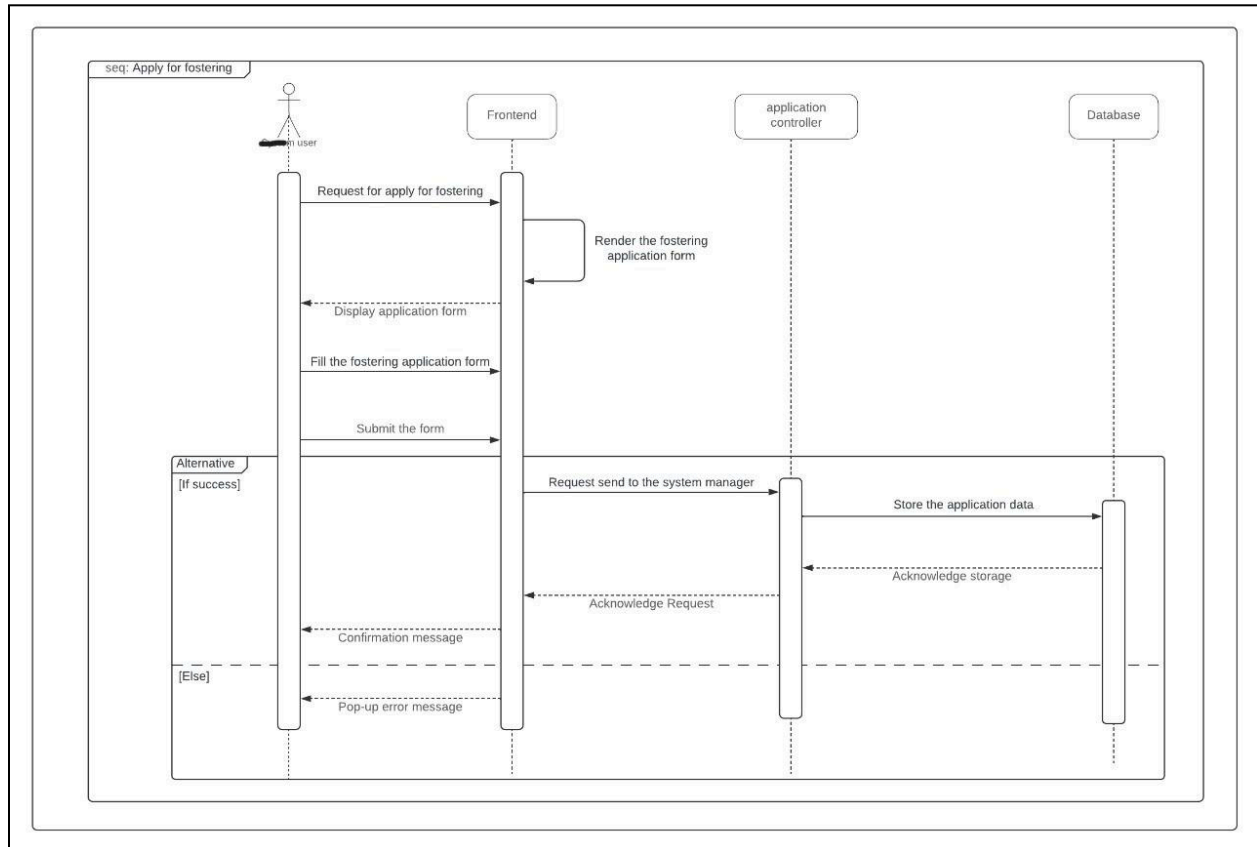
This outlines the process for requesting the creation of a child profile. The orphanage staff starts by requesting to create the profile, and the system routes them to the appropriate form page. After the staff fills in the child's details and submits the request, the system sends the profile details to the orphanage head for approval. The orphanage head accesses the list of profile creation requests, evaluates them, and either approves or rejects the request. If approved, the system creates the child profile. If rejected, the system displays a rejection response to the staff.





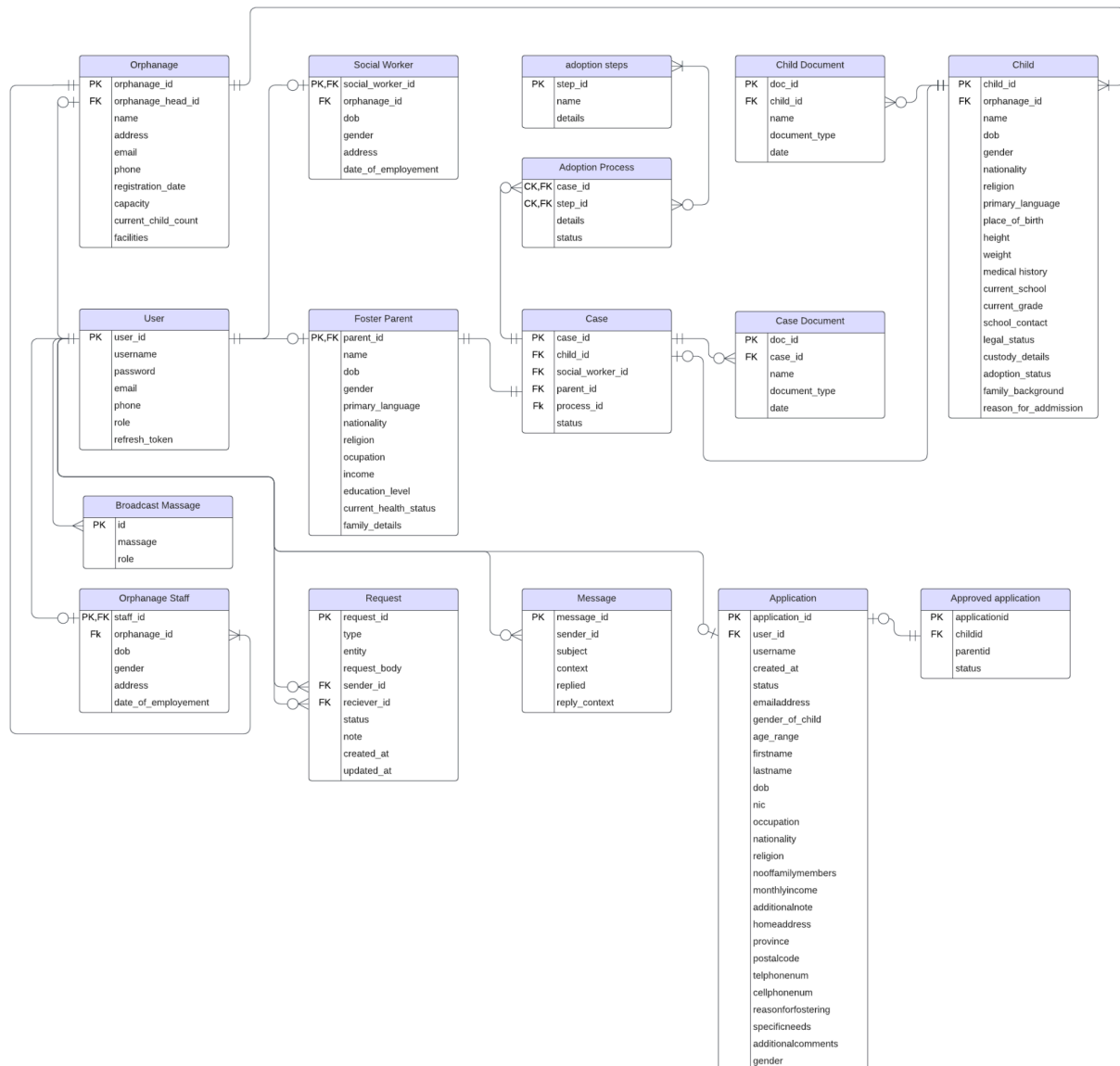
Apply for Fostering

The process for applying for fostering begins with the General Public User logging into the system. After logging in, the user navigates to the fostering section and clicks the "Apply for Fostering" button. The user then fills out the required information in the application form and submits it. Upon submission, the system validates the application. If the submission is successful, a confirmation message is displayed to the user. In case of any issues during the process, an error message is shown, prompting the user to correct the information and try again.



3.3 Database Design

The database schema consists of several interconnected tables designed to manage various aspects of an orphanage management system. Here is the ER diagram for the system.



4. System Implementation

4.1 Implementation Procedure

After finalizing the software design specifications, the business logic and architecture for the OrphanCare system were established. The entire project is a web application that uses various technologies in different situations.

For the frontend, OrphanCare employed a mix of web development tools to create a user-friendly interface. The core framework is React, a JavaScript library that allows the team to build modular and

interactive components, providing a smooth user experience. Tailwind was used for styling and design, enhancing the visual aspects of the application, such as layout, typography, and color schemes, making it both functional and attractive. HTML organized the content logically, while JavaScript added interactivity for user actions and data handling.

The OrphanCare project follows a three-tier architecture, consisting of a presentation layer, a business logic layer, and a data layer. Node.js and Express.js are essential in this setup, with Node.js providing a runtime environment for server-side JavaScript and Express.js serving as a framework for building APIs. This combination efficiently manages requests and server-side logic.

Document storage for OrphanCare is handled using Azure S3, a cloud solution that ensures secure and scalable document storage.

During testing, various tools and frameworks were used, including Jest for unit and component testing, Supertest for API endpoint testing, and Postman for monitoring APIs. These tools ensured thorough testing of both the frontend and backend components.

For deployment, the OrphanCare project utilized Git Actions for continuous integration and deployment (CI/CD), while Docker technologies, along with Docker Hub, facilitated the containerization and efficient deployment of the application.

4.2 Materials

In the OrphanCare project, Google Fonts were utilized for typography, offering a broad range of font styles to enhance the visual appeal of the application. Additionally, icons were sourced from Google's icon library, ensuring a consistent and visually pleasing design. Beyond these, no other external materials were used in the project.

4.3 The Algorithm

```
BEGIN
    INPUT: ageRange, gender

    // Step 1: Calculate the current date
    currentDate = GET_CURRENT_DATE()

    // Step 2: Determine the date range for the age group
    maxDate = currentDate - ageRange[0] years
    minDate = currentDate - ageRange[1] years

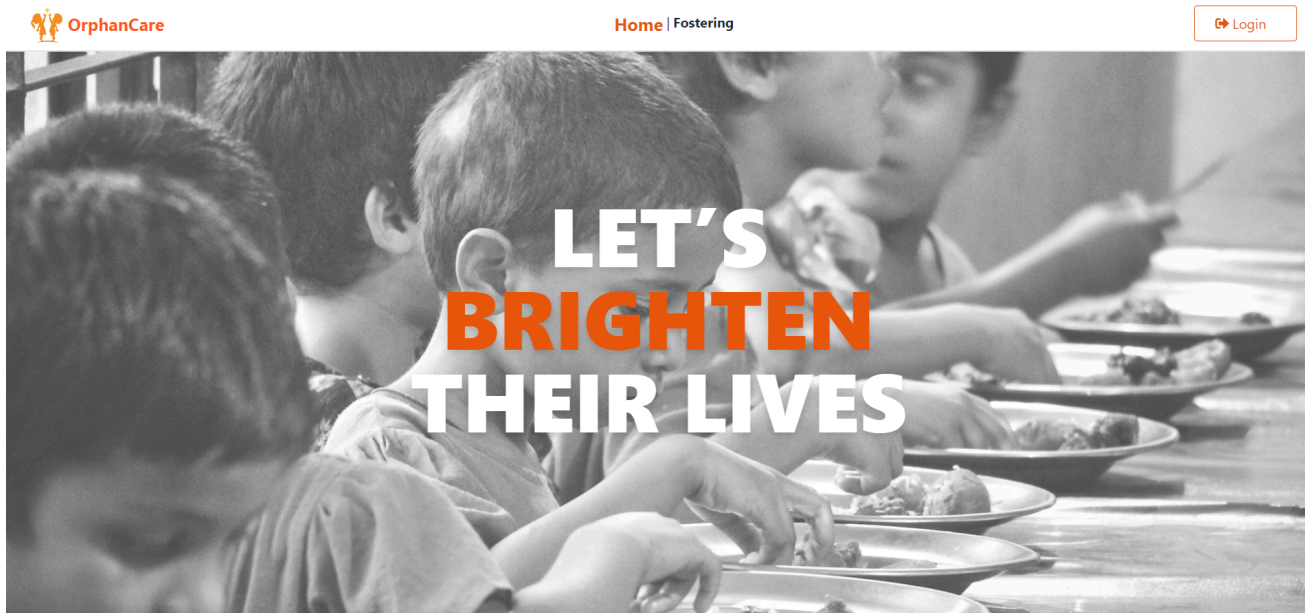
    // Step 3: Query the database for matching children
    childrenList = QUERY_DATABASE(
        WHERE gender IS gender
        AND date_of_birth BETWEEN minDate AND maxDate
        SELECT childID, orphanageID, name, orphanage.orphanageName
    )

    // Step 4: Return the list of matching children
    RETURN childrenList
END
```

4.4 Main Interfaces

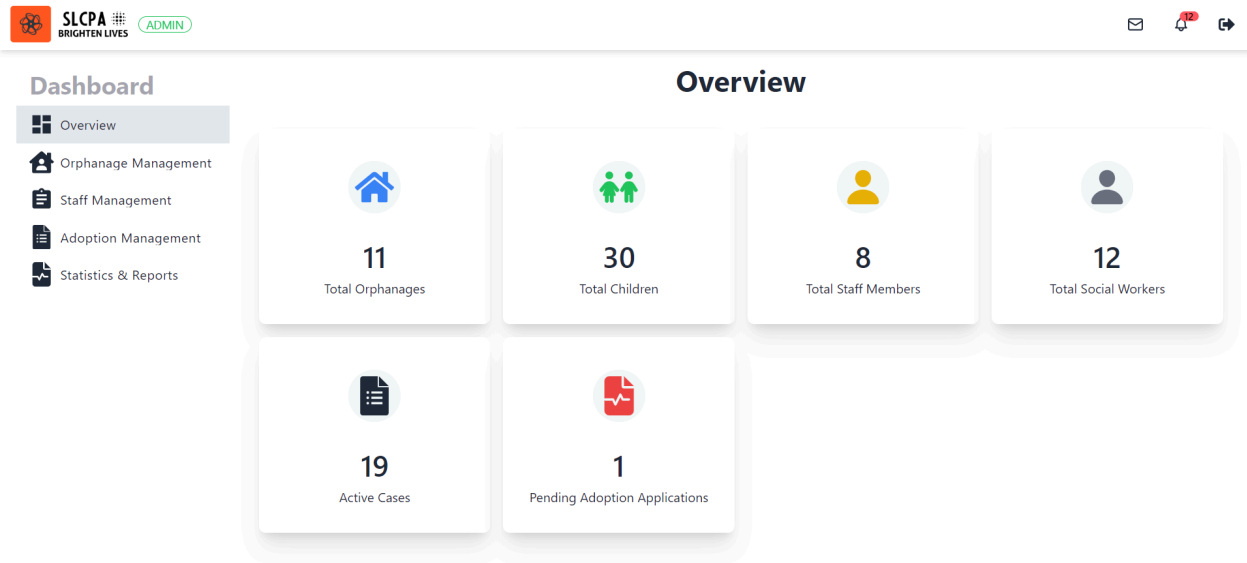
Landing page

This will be the Landing Page for any general user, allowing them to access the page without needing an account.



Admin Dashboard

This will be the Dashboard Page for the Admin. They can access the most important information from the provided window and navigate through different tabs to gain more insights on the orphanage statistics and ongoing activities within the orphanages.



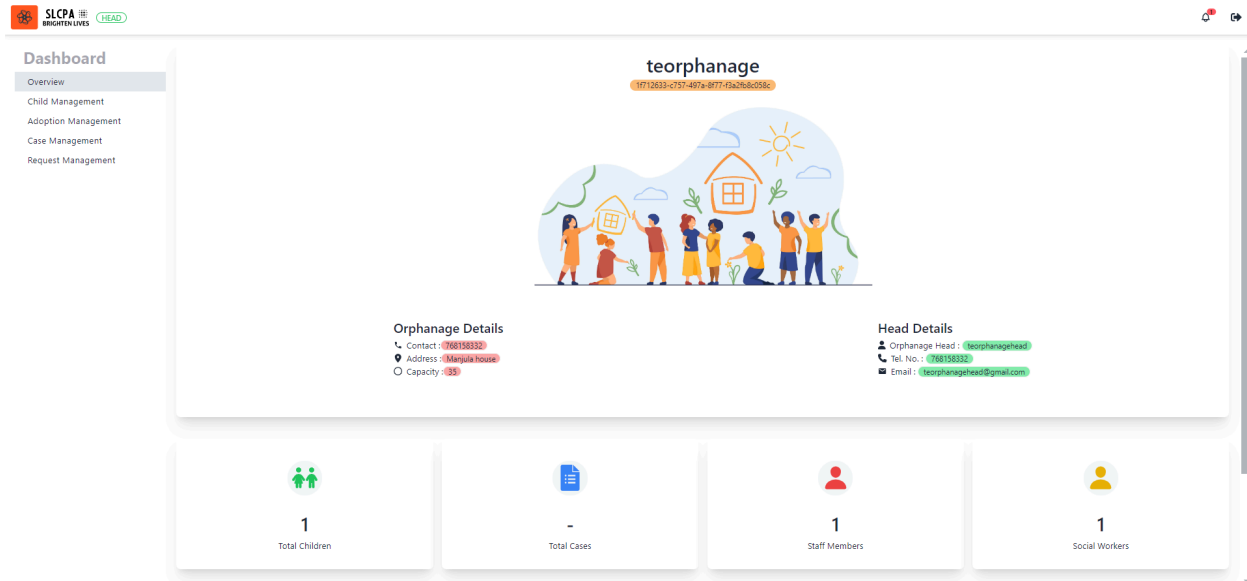
GeneralUser Logged Dashboard

Once a general user wants to adopt a child, they must log in to the system. After logging in, this page will appear, and they can fill out the form by clicking the "Apply Now" button to submit a fostering application.



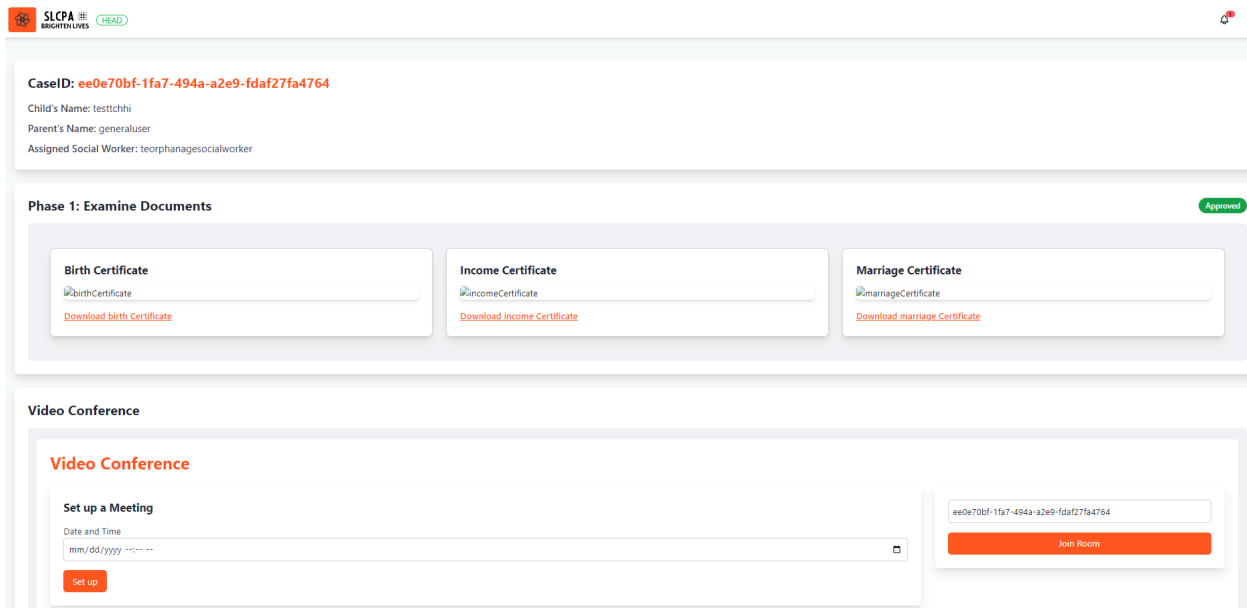
Orphanage Head Dashboard

This is the Dashboard Page for the Head. They can access the most important information about their orphanage from the provided window and navigate through different tabs to gain more insights into the statistics and ongoing activities within their orphanage.



Case Handle Page

Once a fostering case is initiated, this will be the page that the Head and staff can view to manage and continue the rest of the fostering process.



5. System Testing and Analysis

5.1 Testing approach

We utilized a range of testing technologies, including Jest, Vitest, Supertest, and Postman, to ensure our application's robustness. Jest, a flexible JavaScript testing framework, played a key role in conducting unit tests for individual components and functions. Postman's versatility and ease of use made it an essential tool during development, integration, and quality assurance phases, allowing us to send

requests and thoroughly verify responses.

5.2 Unit Testing, Results and analysis of testing

```
at log (controllers/caseController.js:161:13)

PASS tests_/staffTests/addStaff.test.js
  addStaff
    ✓ should successfully create new staff and add to the orphanage (17 ms)
    ✓ should return 500 when an error occurs during the transaction (28 ms)

PASS tests_/caseTests/getCaseById.test.js
  getCaseById
    ✓ should return case details when case is found (8 ms)
    ✓ should return 404 if case is not found (2 ms)
    ✓ should return 500 if prisma query fails (24 ms)

-----
File | % Stats | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files | 38.9 | 92.8 | 37.5 | 38.9 |
config | 100 | 100 | 100 | 100 |
dbConn.js | 100 | 100 | 100 | 100 |
roles_list.js | 100 | 100 | 100 | 100 |
sClient.js | 100 | 100 | 100 | 100 |
controllers | 38.19 | 92.8 | 37.5 | 38.19 |
  applicationController.js | 34.33 | 66.66 | 37.5 | 34.33 | 54-61,63-103,132-170,184-216,229-289,292-317,320-335,338-356
  caseController.js | 38.89 | 90 | 28.57 | 38.89 | 167-213,216-260,265-311,325-354,362-399,401-418,423-436,439-452,455-489,495-547,551-564
  childController.js | 71.97 | 100 | 80 | 71.97 | 123-178
  documentController.js | 100 | 100 | 100 | 100 |
  fileController.js | 20.29 | 90.47 | 14.28 | 20.29 | 65-127,130-161,165-223,226-290,293-360,364-428,431-495,500-519,523-534,537-544,547-565,601-662,673-727
  loginController.js | 30.76 | 71.42 | 100 | 30.76 | 29-106,112-114
  logoutController.js | 100 | 100 | 100 | 100 |
  orphanageController.js | 45.74 | 100 | 44.44 | 45.74 | 8-31,34-62,82-119,149-179,240-270
  refreshTokenController.js | 78.94 | 91.66 | 100 | 78.94 | 31-46
  registerController.js | 81.91 | 81.81 | 100 | 81.91 | 43-56,84-86
  requestController.js | 17.55 | 100 | 13.33 | 17.55 | 46-69,72-94,97-121,124-140,212-284,287-356,359-427,430-509,512-575,578-645,648-710,713-766,769-831
  socialWorkerController.js | 60.31 | 100 | 60 | 60.31 | 8-51,166-179
  staffController.js | 60.76 | 100 | 50 | 60.76 | 36-71,114-128
-----

Test Suites: 29 passed, 29 total
Tests: 91 passed, 91 total
Snapshots: 0 total
Time: 8.294 s
Run all test suites.
PS C:\Users\Valindu Gamage\Desktop\SE Project\Orphanage-Management-System\backend>
```

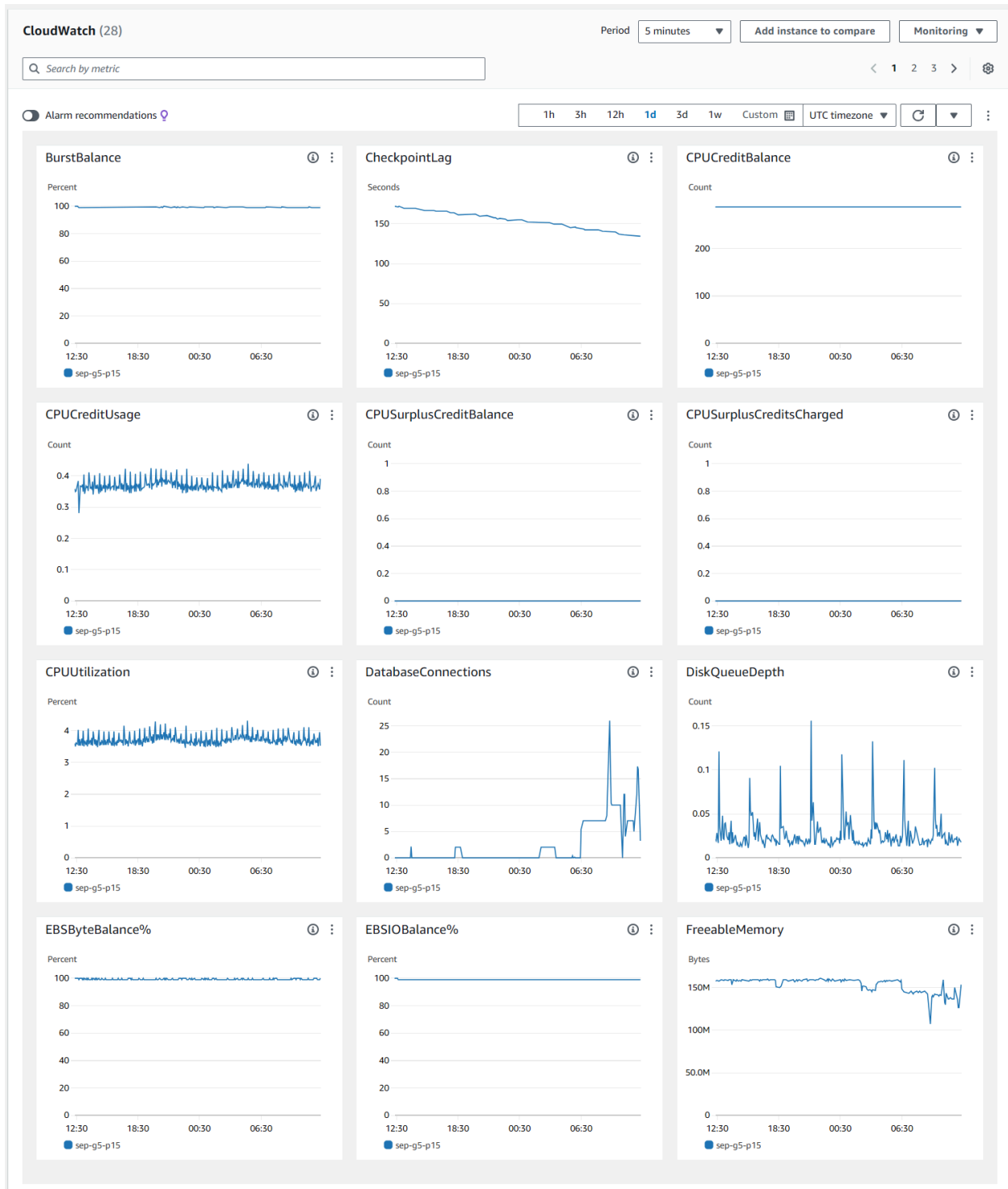
screenshot of the api testing

The test results indicate that various components of the application, including API requests and database operations, are functioning as expected. All **91** tests passed, reflecting stable functionality and well-implemented features across multiple modules, with no failed tests identified. This demonstrates the reliability of the application and suggests that no immediate issues or bugs were detected during testing.

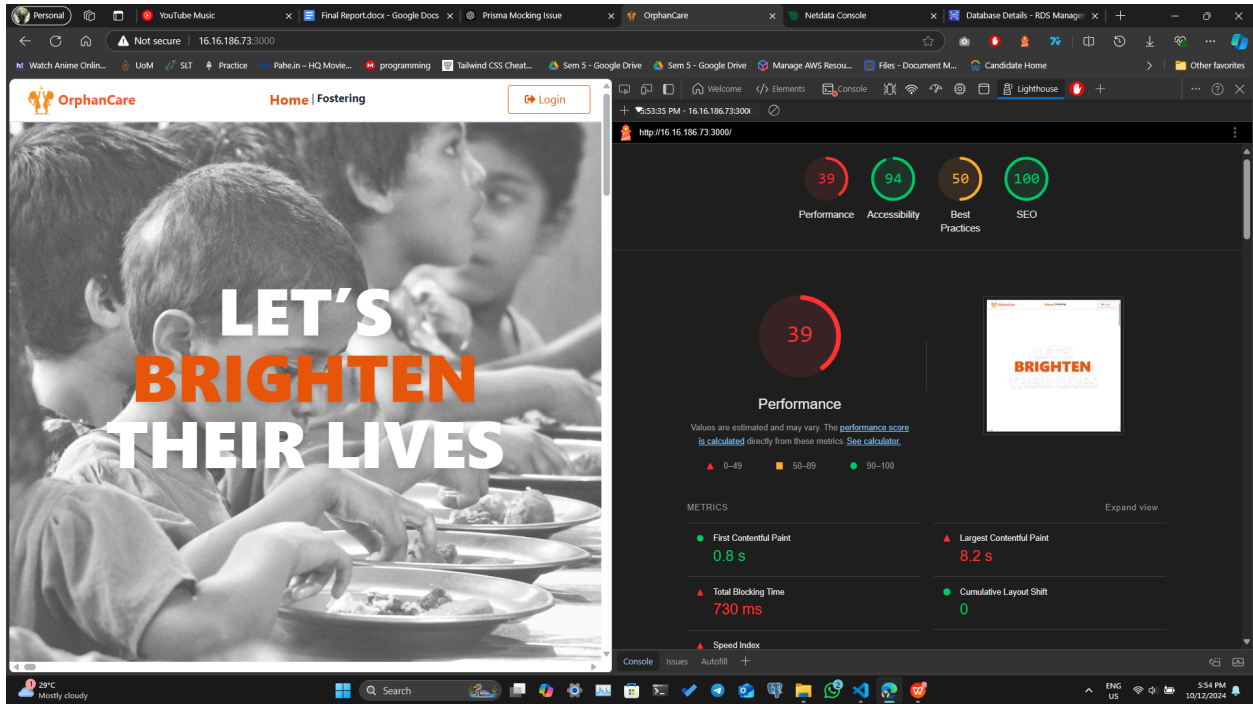
These successful outcomes contribute significantly to the quality assurance process, ensuring that the system meets expected requirements and behaves as intended. The results are a positive sign of the robustness and readiness of the application's features for deployment or further development.

5.3 Aspects related to performance, security, failures

Security and access control testing are vital for ensuring the integrity, confidentiality, and availability of sensitive data within a child protection data privacy management system. This thorough testing covers two key areas: application-level and system-level security. Application-level security ensures that users are restricted to specific functions and data, with access granted based on defined roles and permissions. System-level security manages access to the entire system, allowing only authorized users to pass through secure gateways. By focusing on these critical aspects, organizations can safeguard sensitive information, prevent unauthorized access, and maintain the integrity of child protection data.



performance testing of the database



Browser performance statistics

5.4 System Usability Report

The System Usability Study (SUS) was conducted to evaluate the usability and user-friendliness of the web application OrphanCare, which is designed for managing orphanages under the Child Protection Authority. In this study, external users interacted with the platform and subsequently completed a 10-item questionnaire based on the SUS framework.

We invite you to review the findings of this study through the following links:

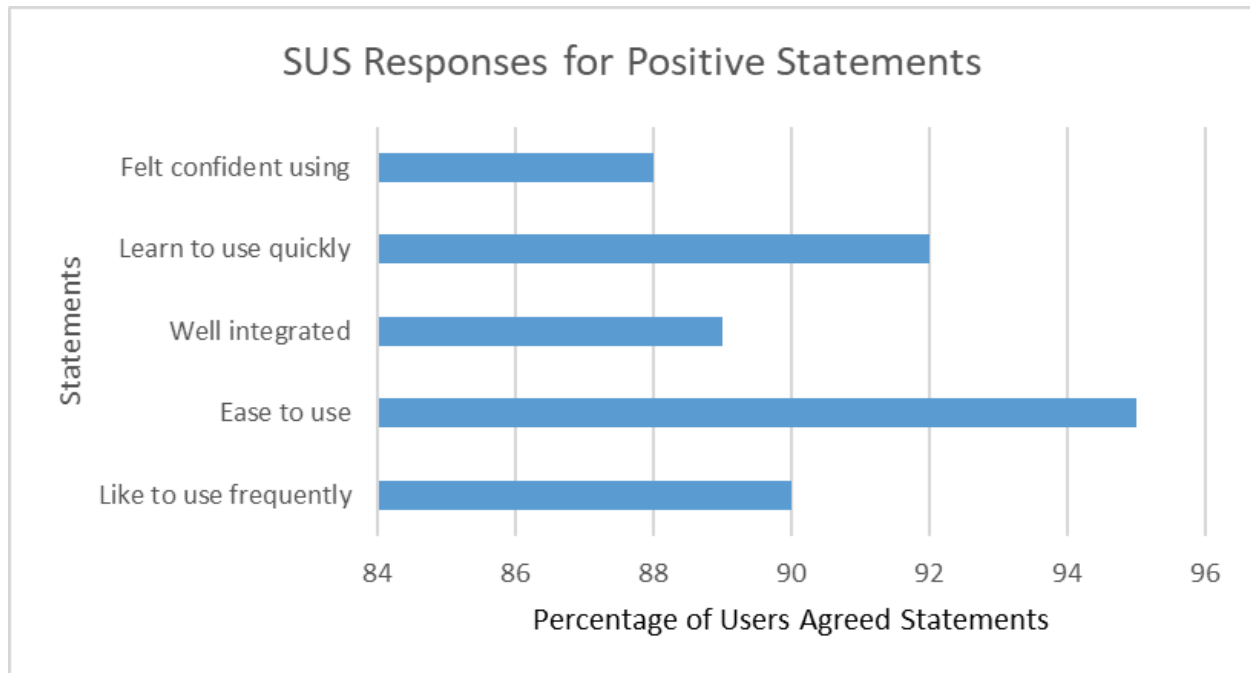
Survey Link for the System Usability Report:

<https://forms.gle/kLRG1xy3FF4AdZBd7>

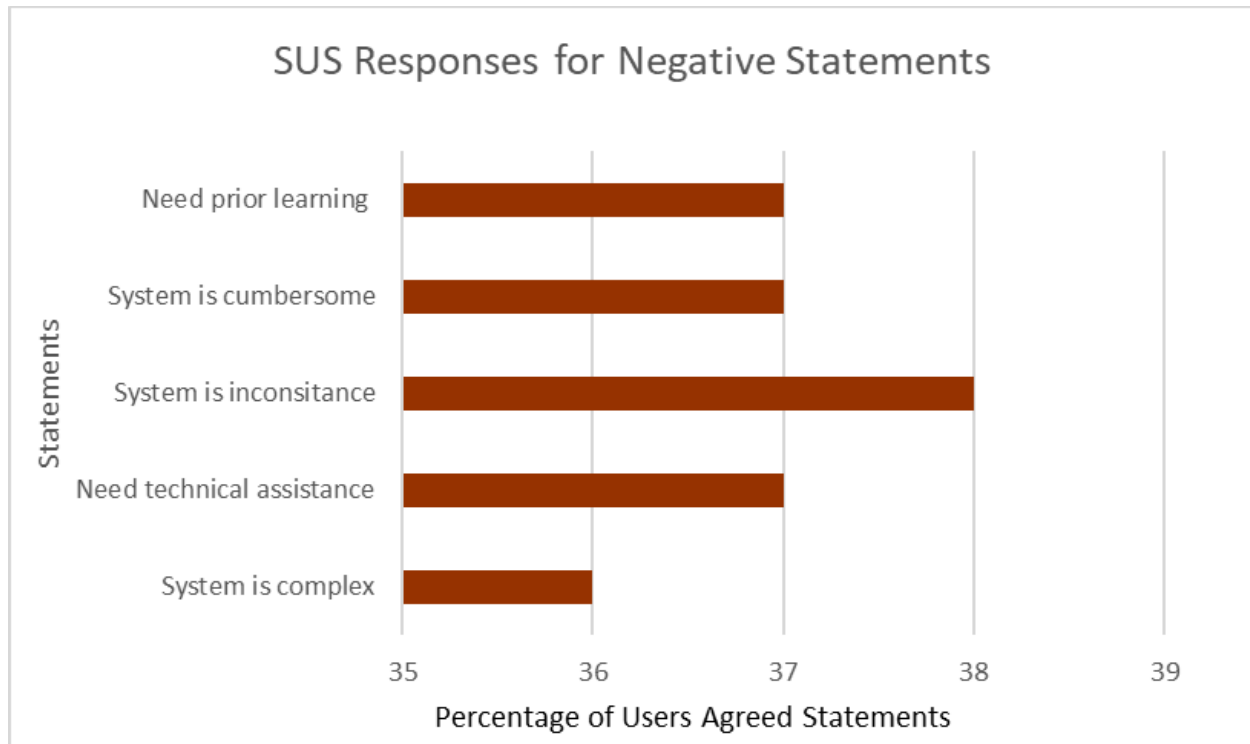
Response Sheet:

https://docs.google.com/spreadsheets/d/1bd8Aqrc3CxTfUPbk9eswQTkMd7ue6kl_6Ri2bcJ9gx0/edit?usp=sharing

Participant	Statement 1	Statement 2	Statement 3	Statement 4	Statement 5	Statement 6	Statement 7	Statement 8	Statement 9	Statement 10	
User 1	5	1	5	1	5	1	5	1	5	1	100.0
User 2	5	2	5	1	4	1	5	1	5	1	95.0
User 3	5	1	5	1	4	1	5	1	5	1	97.5
User 4	5	1	5	1	5	1	5	1	5	1	100.0
User 5	5	1	5	1	5	1	5	1	5	4	92.5
User 6	5	2	5	2	4	2	5	1	4	1	87.5
User 7	5	2	5	1	4	1	5	1	5	1	95.0
User 8	4	1	5	2	5	1	4	1	4	2	87.5
User 9	2	3	4	1	3	2	4	3	3	1	65.0
User 10	4	2	4	3	5	3	5	3	4	2	72.5
User 11	4	1	5	2	4	2	5	1	4	2	85.0
User 12	5	5	5	5	5	5	5	5	5	5	50.0
User 13	5	1	5	1	5	1	5	1	5	1	100.0
User 14	5	4	5	5	5	5	5	4	5	5	55.0
User 15	5	1	5	1	5	1	5	1	5	2	97.5
User 16	5	1	5	1	5	1	4	2	4	1	92.5
User 17	5	1	5	1	5	2	5	1	5	1	97.5
User 18	5	2	4	2	5	1	4	2	3	1	82.5
User 19	3	2	4	2	3	3	3	2	3	2	62.5
User 20	3	2	4	3	3	3	3	4	4	2	57.5
AVERAGE	3	2	4	3	3	3	3	4	4	2	83.625



The System Usability Study (SUS) responses reflect a highly positive user experience with the **OrphanCare** web application. A significant 86% of users felt confident using the platform, highlighting that the system instills trust and comfort in its users. Furthermore, 92% of participants indicated that they could quickly learn to use the platform, demonstrating its intuitive design and minimal learning curve. Additionally, 88% of users agreed that the system is well integrated, suggesting that the different components and features work seamlessly together, enhancing the overall user experience. The ease of use was particularly well-received, with 95% of users affirming that the platform is easy to navigate and interact with. Finally, 92% of users expressed a desire to use the platform frequently, indicating strong user engagement and satisfaction.



The System Usability Study (SUS) responses for negative statements offer insights into areas where the **OrphanCare** web application could be improved. The graph shows that around 36% of users felt that prior learning was necessary to use the platform effectively, indicating that some features may not be immediately intuitive for all users. Additionally, about 36% of respondents found the system to be cumbersome, suggesting that certain tasks may require streamlining to improve the overall user experience. A notable 37% of users felt the system was inconsistent, which could point to a lack of uniformity in design or functionality across different sections of the platform. Moreover, 36% of users stated that they needed technical assistance, which indicates that additional help or guidance may be required for users to navigate certain areas of the system. Lastly, only 35% of users agreed that the system was complex, suggesting that while some found it difficult to navigate, this issue was not as prominent as others.

OrphanCare offers a well-designed and intuitive platform that most users appreciate, but addressing concerns about inconsistency, complexity, and the need for support could further improve the user experience. This balance of strengths and weaknesses indicates that the platform is on the right path, with room for refinement to enhance usability and user satisfaction.

6. Conclusion and Future Work

The main goal of the system is to safeguard and manage children's data while restricting access to authorized users. Existing manual systems lack adequate security measures and are inefficient in handling data breaches. In contrast, the newly developed system offers superior performance by being accessible via a web application, benefiting internal and external users alike. Overall, the system effectively supports orphanage management. In the future, features such as staff leave management and comprehensive financial management could be added.

References

1. Amazon Web Services, "AWS Documentation," [Online]. Available: https://docs.aws.amazon.com/?nc2=h_ql_doc_do [Accessed: 22-Sep-2024].
2. Jest, "Getting Started," [Online]. Available: <https://jestjs.io/docs/getting-started>. [Accessed: 12-Oct-2024].
3. Vitest, "A Vite-native unit test framework," [Online]. Available: <https://vitest.dev/>. [Accessed: 12-Oct-2024].
4. Postman, "Postman API Platform," [Online]. Available: <https://www.postman.com/>. [Accessed: 12-Oct-2024].